



AI 2002 – Artificial Intelligence (Spring 2025) Assignment 2

Topics Covered: Adversarial Search (Minimax, Alpha-Beta Pruning), Local Searches (Hill Climbing, Simulated Annealing, Local Beam Search), Heuristic Evaluation Functions, Game Playing AI

Submission Deadline: Monday – March 17, 2025 by 23.00 sharp

Submission Guidelines:

- You must **understand every line of code** you submit.
- Submit the **code** for **Questions 1 & 3** and a **PDF** report for **Questions 2 & 4** on Google Classroom.
- **Plagiarism will not be accepted.**

Implementing and Analyzing Adversarial Search Algorithms in 4x4 Tic-Tac-Toe and Checkers

Overview:

This assignment focuses on adversarial search in the context of two games: **4x4 Tic-Tac-Toe** and **Checkers**, along with **Local Search Algorithms** for solving the **8-Queens Problem**. You will implement and analyze AI agents using Minimax, Alpha-Beta Pruning, and Local Search strategies.

The boilerplate code for both games and the 8-Queens problem has been provided. Your task is to implement the adversarial search algorithms for the games and a **step-by-step visualization** of local search techniques for the **8-Queens Problem**.

Question 1: 4x4 Tic-Tac-Toe AI

Implement an AI for 4x4 Tic-Tac-Toe using the following search methods:

1. **Minimax Algorithm** (without pruning)
2. **Minimax with Alpha-Beta Pruning**

Implementation Requirements:

- Modify the given code so that the user can play against the AI.
- Allow the user to choose whether AI plays first or second.
- Display the game state at each step.

Performance Analysis:

- Count the number of nodes expanded in both methods.
 - Measure execution time for both approaches.
 - Compare results when AI plays first vs. second.
-

Question 2: Tic-Tac-Toe AI Analysis

Perform a comparative analysis based on your Tic-Tac-Toe implementation:

- Create a table listing the number of nodes expanded and execution time.
 - Which method performed better in terms of speed?
 - How much reduction in node expansion did Alpha-Beta Pruning provide?
 - Does playing first or second affect AI performance? Explain why.
-

Question 3: Checkers AI

Implement an AI for Checkers using the following strategies:

1. **Minimax with depth-limited search** (choose a reasonable depth limit)
2. **Minimax with Alpha-Beta Pruning**

Implementation Requirements:

- Allow the user to specify the AI's depth limit.
- Display the board state after each move.
- Allow the user to play against the AI.

Performance Analysis:

- Compare performance at different depth levels.
 - Measure execution time and node expansions.
 - Analyze how pruning impacts search efficiency.
-

Question 4: Checkers AI Analysis

Perform a comparative analysis based on your **Checkers** implementation:

- Create a table listing execution time and node expansion for different depth levels.
 - How does increasing depth affect search efficiency?
 - How much improvement does Alpha-Beta Pruning provide?
 - Discuss the trade-offs between depth and AI decision quality.
-

Question 5: 8 Queen Problem using local search

Implement local search algorithms to solve the **8-Queens Problem** using all of the following techniques:

1. **Hill Climbing**
2. **Simulated Annealing**
3. **Local Beam Search**

Implementation Requirements:

- Modify the provided graphical boilerplate code to implement all three local search techniques.
- Ensure the AI attempts to minimize the number of queen conflicts.
- Implement **step-by-step visualization** to show how the queens move across the board during the search.
- Compare the effectiveness of each approach in solving the problem.
- Display the final solved board once the algorithm converges.

Question 4: 8 Queen Analysis

Perform a comparative analysis based on your **8 Queen** implementation:

- Compare the effectiveness of **Hill Climbing, Simulated Annealing, and Local Beam Search** in solving the 8-Queens problem.
- Which method found the solution the fastest?
- How does randomness in Simulated Annealing impact the solution process?
- Explain how the visualization helps understand the search behavior.
- Which local search algorithm is more likely to escape local optima, and why?

Bonus Challenge (Optional)

- Implement a heuristic evaluation function for Checkers to improve AI performance at greater depths. Experiment with different heuristics and compare results.
- Implement **multiple restarts** for Hill Climbing to escape local maxima.
- Compare the efficiency of restarts vs. Simulated Annealing in finding a solution.

Deliverables:

- **Code Submission:** Python implementations for Tic-Tac-Toe, Checkers AI and 8 Queen.
- **Report Submission:** Analysis including tables, comparisons, and explanations.
- **Bonus (if attempted):** Additional heuristics, performance comparisons and multiple restarts or hybrid approaches.

Best of Luck 😊